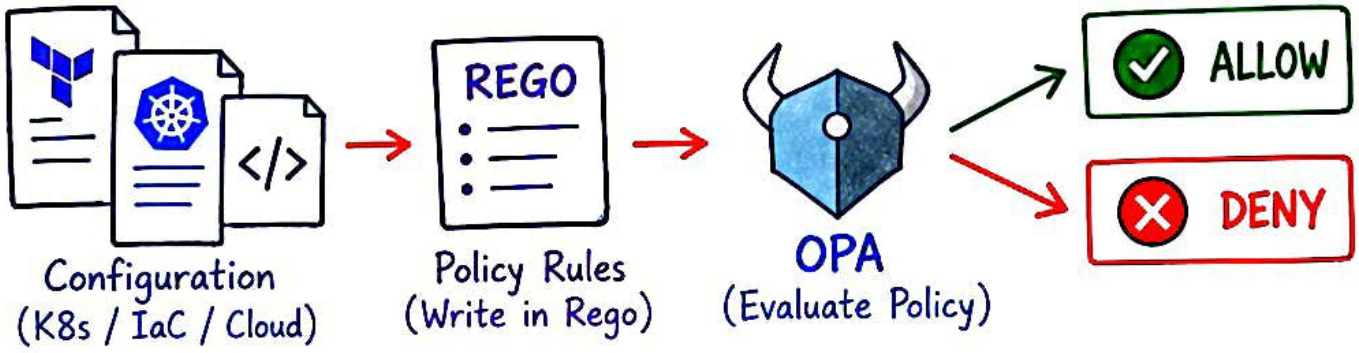
NEERAJ SINGH
DEVOPS

Slide 1/6

REGO IN CLOUD SECURITY

Policy as Code Made Simple



WRITE Security Rules

CHECK Configuration

BLOCK Violations

ENFORCE Compliance

DEFINE → EVALUATE → ALLOW / DENY → ENFORCE

WHAT IS REGO?

Rego is the declarative policy language used by **Open Policy Agent (OPA)** to define and evaluate policies.

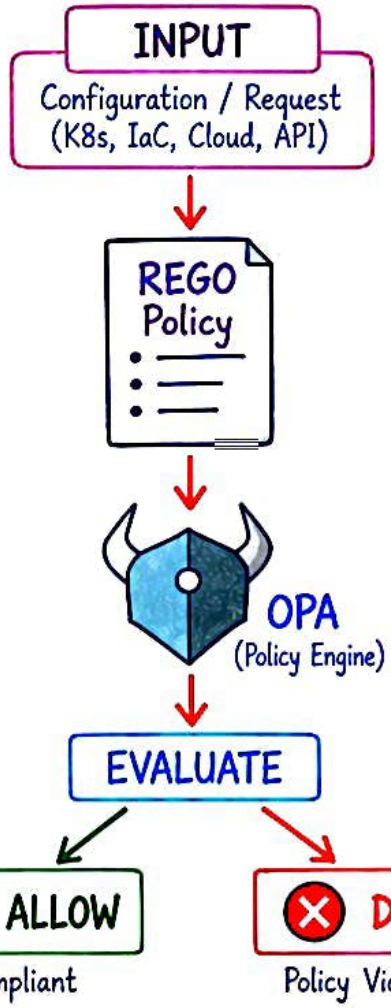
Key Points

- Declarative policy language
- Used with Open Policy Agent (OPA)
- Defines rules to allow or deny requests
- Widely used in cloud, Kubernetes, CI/CD and infrastructure security

Rego vs OPA

Rego → Policy Language (Write the rules)

OPA ⇒ Policy Engine (Evaluates the rules)



Common Use Cases

- Kubernetes Admission Control
- Cloud Security (AWS / Azure / GCP)
- Infrastructure as Code (Terraform, CloudFormation)
- CI/CD Pipeline Security Gates
- API Authorization and Access Control
- Compliance & Governance

WRITE RULES ONCE → EVALUATE AUTOMATICALLY → ENFORCE EVERYWHERE

REGO – HOW DOES IT WORK?

Scenario: Privileged containers are NOT allowed

1. Rego Policy

```
package kubernetes.security

deny[msg] {
    input.spec.containers[_]
     .securityContext.privileged == true

    msg := "Privileged containers
            are not allowed"
}
```

2. Kubernetes YAML (Input)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: nginx
    securityContext:
      privileged: true
```

3. Evaluation Flow



Kubernetes YAML (Input)

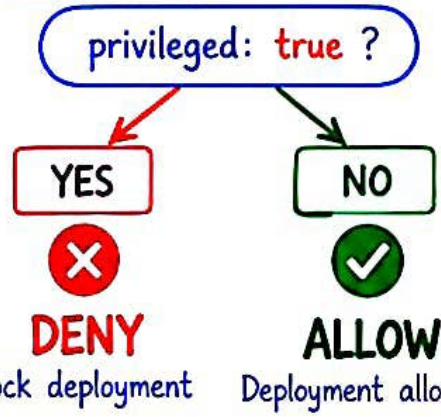


OPA (Policy Engine)



Evaluate against rules

4. Decision

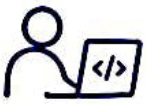


POLICY VIOLATION → STOP BEFORE DEPLOYMENT

REGO IN KUBERNETES

Use Rego with OPA Gatekeeper to enforce security policies in Kubernetes

How it Works?



Developer

↓

kubectl / CI/CD

↓

Kubernetes API Server

↓

Admission Controller

↓

OPA Gatekeeper
(Uses Rego Policy)



↓

Policy Evaluation

↙ ↘

ALLOW
Resource Created

DENY
Request Blocked

Rego Policies in Kubernetes



Block privileged containers



Allow only approved container registries



Require resource requests & limits



Enforce mandatory labels



Restrict host networking and hostPath



Ensure non-root containers



Enforce organizational security standards

Components

REGO
Policy Language
(Write rules)



OPA
Policy Engine
(Evaluate rules)



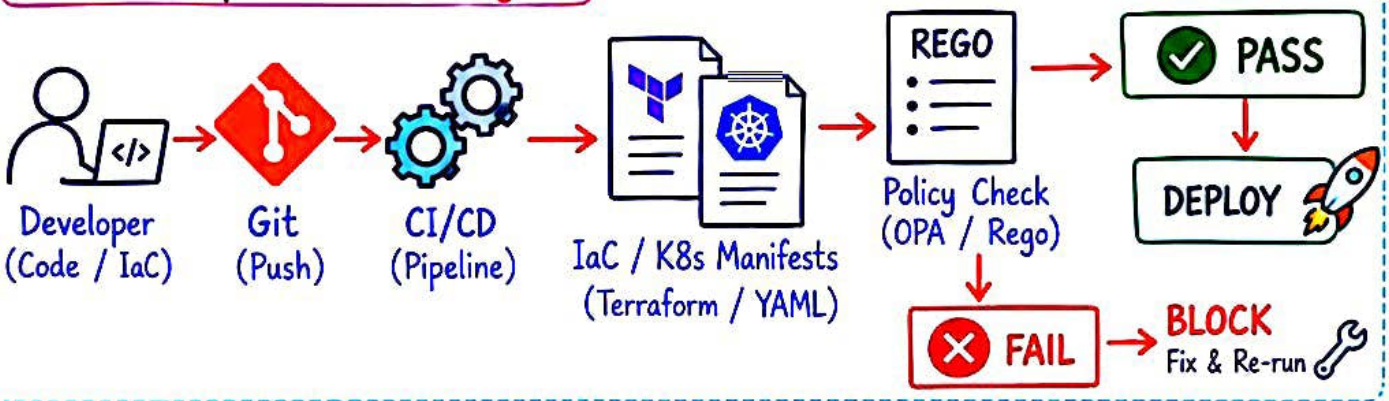
GATEKEEPER
Kubernetes Policy Enforcement
(Uses Rego)

KUBERNETES REQUEST → CHECK POLICY → ALLOW / DENY

REGO IN DevSecOps & CI/CD

Automate security & compliance checks using Rego in your pipelines

CI/CD Pipeline with Rego



Example Policies (Cloud Security)

- No public storage (S3)
- Encryption must be enabled
- Mandatory resource tags
- Use approved container registries
- Restrict admin access
- Follow compliance standards (CIS, SOC2)

Benefits

- Automated security checks
- Prevent misconfigurations early
- Enforce organizational policies
- Shift-left security
- Consistent compliance across environments
- Faster feedback to developers
- Reduces human errors
- Works for multi-cloud, Kubernetes & IaC

POLICY AS CODE → DETECT EARLY → SECURE DEPLOYMENTS

REGO - QUICK REVISION

REGO



Policy language used to write security rules.

OPA



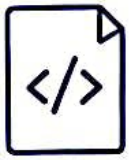
Policy engine that evaluates Rego policies.

GATEKEEPER



Kubernetes policy enforcement engine built on OPA.

POLICY AS CODE



Security rules stored, reviewed and automated like code.

Key Takeaways

- Rego is a declarative policy language
- Used with Open Policy Agent (OPA.)
- Define rules to allow or deny requests
- Used in Kubernetes, CI/CD, Cloud & IaC
- Helps enforce security and compliance
- Automates policy checks across environments

Common Use Cases

Kubernetes Admission Control	Cloud Security	Infrastructure as Code	CI/CD Pipelines
Compliance & Governance	Access Control	Block Risky Configurations	Approved Container Registries

Interview Quick Check

Q: What is Rego?
A: Rego is the declarative policy language used by Open Policy Agent (OPA).

Q: What is OPA?
A: OPA is a general-purpose policy engine that evaluates Rego policies.

Q: What is Gatekeeper?
A: Gatekeeper is a Kubernetes policy enforcement engine built on OPA.

Q: Why use Rego in DevSecOps?
A: To automate security, authorization and compliance decisions.

Final Memory

REGO → OPA → GATEKEEPER

Write the rules → Evaluate the rules → Enforce Kubernetes policies

DEFINE → EVALUATE → ALLOW / DENY → ENFORCE